

Institute of Business Administration
Karachi, Pakistan

Computer Architecture and Assembly Language

Kalman Filter: Milestone 4

RISC-V Vector (RVV) Assembly Implementation

Team Warjingo

Muhammad Ismail	31689
Mehdi Ali	30507
Muhammad Ismail	30917
Arham Awan	30934

May 2026 — Institute of Business Administration, Karachi

Abstract

This report documents the Milestone 4 implementation of RISC-V Vector (RVV 1.0) extensions for both the Linear Kalman Filter (LKF) and Extended Kalman Filter (EKF) used in a 23-joint, 276-dimensional 3D human gait tracking system. Building directly on the scalar assembly implementations from Milestone 3, the core computational kernels—`zero_mem`, `mat_add`, `mat_sub`, `mat_transpose`, `mat_mul`, and `lu_solve`—are replaced with RVV instructions operating on 256-bit vector registers ($VLEN = 256$, element width `e64`, $VLMAX = 4$ doubles per register for $LMUL = m1$).

Both vectorised implementations are verified against the Milestone 3 scalar assembly outputs across all 3040 timesteps and 276 state dimensions. Maximum absolute error is exactly 0.0 for both filters, confirming bit-identical numerical output guaranteed by the fused nature of `vfmac`, which matches the IEEE 754 rounding of the scalar `fmadd.d` used in Milestone 3.

Quantitative instruction-count analysis yields a $3.43\times$ reduction in the `mat_mul` inner loop and a $3.33\times$ reduction for `mat_add/mat_sub`. Applying Amdahl’s Law predicts a hardware speedup of $\approx 2.7\times$ for the LKF and $\approx 2.1\times$ for the EKF. Observed QEMU 8.2 wall-clock times are $4.3\times$ and $3.7\times$ slower than scalar, consistent with the emulator’s element-sequential simulation of vector instructions.

Contents

1	Introduction	3
1.1	Source Files	3
1.2	Build and Execution	3
2	RISC-V Vector Extension Overview	4
2.1	Configuration Used	4
2.2	Instructions Used in This Milestone	4
3	Design Choices	4
3.1	Handle-Table Architecture (Inherited from Milestone 3)	4
3.2	LMUL = m1: Conservative But Correct	4
3.3	Tail Handling	5
4	Function-by-Function Walkthrough	5
4.1	zero_mem(ptr, count) — Vectorised Memory Zeroing	5
4.2	mat_add and mat_sub — Vectorised Element-wise Arithmetic	5
4.3	mat_transpose — Vectorised Transpose with Strided Store	6
4.4	mat_mul — Vectorised Matrix Multiplication	6
4.5	EKF-Specific Functions (Scalar, Unchanged)	7
4.6	lu_solve — Vectorised LU Decomposition	8
5	Matrix Inversion Strategy	9
6	Numerical Verification	9
6.1	Table 1: Average Absolute Error per Joint	10
6.2	Table 2: Average Absolute Error per State Component	10
6.3	Table 3: Global Max/Min Absolute Error	10
6.4	Table 4: M3 Scalar vs M4 Vector Error (Selected Joints)	10
6.5	Why the Error Is Exactly Zero	10
7	Performance Analysis	11
7.1	Instruction Count Reduction	11
7.1.1	Scalar Instruction Count Model	11
7.1.2	Vector Instruction Count Model	11
7.1.3	Per-Function Reduction Table	12
7.2	Theoretical Hardware Speedup (Amdahl's Law)	12
7.3	Observed QEMU Timing	12
7.3.1	Why QEMU Is Slower for Vector Code	12
7.4	Performance Visualisation	13
8	Memory and Alignment	14
9	Instruction Usage Summary	14
10	Conclusion	14

1 Introduction

Milestone 4 is the final stage of the Kalman Filter project. The objective is to replace the scalar RISC-V floating-point loops from Milestone 3 with RISC-V Vector (RVV 1.0) instructions, achieving data parallelism inside the same 276-dimensional filter pipeline.

The system remains unchanged from Milestones 2 and 3:

$$N = 12 \times 23 = 276, \quad M = 3 \times 23 = 69, \quad \Delta t = 0.01 \text{ s}, \quad \sigma_j = 1.0, \quad \sigma_r = 0.5$$

$$F \in R^{276 \times 276}, \quad Q \in R^{276 \times 276}, \quad H \in R^{69 \times 276}, \quad R \in R^{69 \times 69}$$

1.1 Source Files

- `lkf_vector.s` — Vectorised LKF: all math kernels use RVV; filter loop and handle-table layout identical to Milestone 3. 1435 lines.
- `ekf_vector.s` — Vectorised EKF: same vectorised math kernels; scalar code retained for `approx_atan2`, `state_to_spherical`, `meas_to_spherical`, and `compute_jac` (inherently sequential). 1711 lines.
- `lkf_helpers.c` / `ekf_helpers.c` — Unchanged C helpers for CSV I/O. No computation in C.
- `verify_m4.c` — Verification tool: compares M4 vector against M3 scalar output, reporting all four required error tables.
- `Makefile` — Builds scalar (M3) and vector (M4) targets separately; provides `run_vector`, `benchmark`, and `run` targets.

1.2 Build and Execution

```

1 # Assemble with RVV support
2 riscv64-linux-gnu-as -march=rv64gcv -o lkf_vector.o lkf_vector.s
3
4 # Link (FIX 2: helper object is lkf_helpers.o, not lkf_io.o)
5 riscv64-linux-gnu-gcc -o lkf_vector lkf_vector.o lkf_helpers.o -static -lm
6
7 # Execute under QEMU 8.2 with VLEN=256
8 ~/qemu-8.2.2/build/qemu-riscv64 -cpu rv64,v=on,vlen=256 ./lkf_vector
9
10 # Full pipeline
11 make run_vector      # build and run both vector filters
12 ./verify_m4         # compare against M3 scalar output
13 make benchmark      # timed comparison M3 vs M4

```

Listing 1: Build and run commands

QEMU Version Note. The Ubuntu stock QEMU 6.2 does not expose a functional RVV unit (`vsetvli` returns `v1=0`). QEMU 8.2 was built from source and is invoked via `-cpu rv64,v=on,vlen=256`.

2 RISC-V Vector Extension Overview

2.1 Configuration Used

Parameter	Value
VLEN (hardware register width)	256 bits
ELEN (max element width)	64 bits
Element type	e64 (IEEE 754 double)
Grouping (LMUL)	m1 (single register)
VLMAX	256/64 = 4 doubles per register
Tail policy	ta (tail agnostic)
Mask policy	ma (mask agnostic)

2.2 Instructions Used in This Milestone

Class	Instructions	Used In
Configuration	<code>vsetvli</code>	All vectorised functions
FP broadcast	<code>vfmv.v.f</code>	<code>zero_mem</code> , <code>mat_mul</code> (zero C)
Unit-stride load	<code>vle64.v</code>	All arithmetic kernels
Unit-stride store	<code>vse64.v</code>	All arithmetic kernels
Strided store	<code>vsse64.v</code>	<code>mat_transpose</code>
FP add (vv)	<code>vfadd.vv</code>	<code>mat_add</code>
FP sub (vv)	<code>vfsb.vv</code>	<code>mat_sub</code>
Fused multiply-acc vf	<code>vfmacc.vf</code>	<code>mat_mul</code> inner j -loop
Fused neg-mult-acc vf	<code>vfnmsac.vf</code>	<code>lu_solve</code> elimination & fwd/back sub
FP division	<code>vfdv.vf</code>	<code>lu_solve</code> back-substitution

3 Design Choices

3.1 Handle-Table Architecture (Inherited from Milestone 3)

The handle-table design from Milestone 3 is carried over unchanged. Register `s3` holds the base of a heap-allocated pointer table with 33 entries (33×8 bytes). All matrix pointers are loaded with a single `ld` instruction:

```

1 .equ PT_F,    0      # F matrix (276x276)
2 .equ PT_P,   48      # covariance P (276x276)
3 .equ PT_K,  144      # Kalman gain K (276x69)
4 # ...
5 ld a0, PT_F(s3)     # load F pointer into a0

```

Listing 2: Handle-table pointer loading

3.2 LMUL = m1: Conservative But Correct

The implementation uses `m1` (single-register grouping) throughout rather than larger groupings (`m4` or `m8`). With $VLEN = 256$ this gives $VLMAX = 4$ doubles per operation. This choice was made for two reasons:

1. **Register pressure.** Using `m8` would occupy 8 consecutive vector registers per operand; three operands (A, B, C in `mat_mul`) would require 24 of the 32 vector registers, leaving no free registers for intermediate values. With `m1`, only `v0`, `v1`, `v2` are needed.

2. **Portability.** A VLEN-agnostic implementation with `m1` runs correctly on any hardware with $VLEN \geq 64$ bits. The `vsetvli` tail-agnostic policy handles any remainder automatically. On hardware with $VLEN=512$ the same code automatically uses $VLMAX=8$ doubles per operation, delivering further speedup without source changes.

3.3 Tail Handling

For array lengths that are not multiples of $VLMAX=4$, the standard idiom:

```
1 vsetvli t2, remaining, e64, m1, ta, ma # vl = min(remaining, 4)
```

is used inside every loop. When $remaining < 4$, the hardware sets $vl = remaining$ and only that many elements are processed. This eliminates all special-case boundary code.

4 Function-by-Function Walkthrough

4.1 zero_mem(ptr, count) — Vectorised Memory Zeroing

Replaces the scalar byte-by-byte loop with a broadcast-and-store pattern. `vfmv.v.f` broadcasts the scalar 0.0 (from `ft0`) into all vl lanes of `v0`; `vse64.v` then stores all lanes in a single instruction.

```
1 zero_mem:
2     fcvt.d.w   ft0, zero           # ft0 = 0.0
3     li        t0, 0               # i = 0
4 zm_loop:
5     bge       t0, a1, zm_done
6     sub       t1, a1, t0           # remaining = count - i
7     vsetvli   t2, t1, e64, m1, ta, ma # vl = min(remaining, 4)
8     vfmv.v.f  v0, ft0             # v0[0..vl-1] = 0.0
9     slli      t3, t0, 3
10    add       t4, a0, t3
11    vse64.v   v0, (t4)             # store vl zeros to memory
12    add       t0, t0, t2           # i += vl
13    j         zm_loop
14 zm_done:
15    ret
```

Listing 3: zero_mem: vectorised fill with 0.0

Register usage: `ft0` (0.0 constant), `t0`–`t4` (loop counters and address temporaries), `v0` (zero vector). No callee-saved registers used; no stack frame needed.

4.2 mat_add and mat_sub — Vectorised Element-wise Arithmetic

Both functions share the same loop structure, differing only in `vfadd.vv` vs. `vfsb.vv`.

```
1 mat_add:
2     li        t0, 0
3 ma_loop:
4     bge       t0, a3, ma_done
5     sub       t1, a3, t0
6     vsetvli   t2, t1, e64, m1, ta, ma
7     slli      t3, t0, 3
8     add       t4, a0, t3           # &A[i]
9     add       t5, a1, t3           # &B[i]
10    add       t6, a2, t3           # &C[i]
```

```

11     vle64.v    v0, (t4)           # load A chunk
12     vle64.v    v1, (t5)           # load B chunk
13     vfadd.vv   v2, v0, v1         # C = A + B (element-wise, VL elems)
14     vse64.v    v2, (t6)           # store C chunk
15     add        t0, t0, t2         # i += vl
16     j          ma_loop

```

Listing 4: mat_add: vectorised element-wise addition

These two functions handle all 276-element vector operations (e.g., $\hat{x}_k = \hat{x}_p + Ky$) and all 76 176-element matrix operations (e.g., $P_{\text{pred}} = FPF^T + Q$ and $P_k = (I - KH)P_p(I - KH)^T + KKK^T$).

4.3 mat_transpose — Vectorised Transpose with Strided Store

This is a notable improvement over the Milestone 3 scalar implementation. Each row of the source matrix is loaded contiguously with `vle64.v` and then scattered into the destination column using `vsse64.v` with `stride = n_rows × 8` bytes.

```

1 mat_transpose:
2     # a0=src, a1=dst, a2=nrows, a3=ncols
3     slli        a4, a2, 3          # stride = nrows * 8 bytes
4     li          t0, 0              # i = 0 (row)
5 mt_outer:
6     bge         t0, a2, mt_done
7     li          t1, 0              # j = 0 (col)
8 mt_inner:
9     bge         t1, a3, mt_next_row
10    sub          t2, a3, t1
11    vsetvli      t3, t2, e64, m1, ta, ma
12    mul          t4, t0, a3
13    add          t4, t4, t1
14    slli         t4, t4, 3
15    add          t5, a0, t4
16    vle64.v      v0, (t5)           # load A[i][j..j+vl-1]
17    mul          t4, t1, a2
18    add          t4, t4, t0
19    slli         t4, t4, 3
20    add          t5, a1, t4
21    vsse64.v     v0, (t5), a4        # scatter: B[j][i], B[j+1][i], ...
22    add          t1, t1, t3
23    j            mt_inner

```

Listing 5: mat_transpose: contiguous load + strided column store

The `vsse64.v` instruction writes `vl` elements to addresses `base`, `base + stride`, ..., `base + (vl - 1) · stride`, where `stride = n_rows × 8`. This maps each source row element $A[i][j]$ to the correct transposed position $B[j][i]$ in a single vector operation.

4.4 mat_mul — Vectorised Matrix Multiplication

The most performance-critical function, called ≈ 15 times per timestep or $\approx 45\,000$ times per filter run. The `ikj` loop order is retained from Milestone 3, with the innermost `j`-loop replaced by a `vfmacc.vf` vector operation. The zero-skip optimisation (skip the entire `j`-loop if $A[i][k] = 0$) is preserved to exploit sparsity in F and H .

```

1     # --- Zero C (vectorised) ---
2     mul         t0, s3, s5          # total elements = rowsA * colsB
3     fcvtd.w     ft3, zero

```

```

4      li          t1, 0
5 mmz_loop:
6      sub         t2, t0, t1
7      vsetvli     t3, t2, e64, m1, ta, ma
8      vfmv.v.f    v0, ft3
9      slli        t4, t1, 3
10     add         t5, s2, t4
11     vse64.v     v0, (t5)
12     add         t1, t1, t3
13     blt         t1, t0, mmz_loop
14
15     # --- ikj triple loop ---
16     li          t0, 0                # i
17 mm_i:
18     bge         t0, s3, mm_done
19     li          t1, 0                # k
20 mm_k:
21     bge         t1, s4, mm_next_i
22     fld         ft0, [A + i*colsA + k]*8 # ft0 = A[i][k]
23     feq.d       a6, ft0, ft7
24     bnez        a6, mm_next_k        # zero-skip
25     li          t6, 0                # j = 0
26 mm_j:
27     bge         t6, s5, mm_next_k
28     sub         a6, s5, t6
29     vsetvli     a7, a6, e64, m1, ta, ma # vl = min(remaining, 4)
30     vle64.v     v1, [B + k*colsB + j]*8 # v1 = B[k][j..j+vl-1]
31     vle64.v     v2, [C + i*colsB + j]*8 # v2 = C[i][j..j+vl-1]
32     vfmacc.vf   v2, ft0, v1          # C += A[i][k]*B[k][j..] (fused)
33     vse64.v     v2, [C + i*colsB + j]*8 # store
34     add         t6, t6, a7
35     j           mm_j

```

Listing 6: mat_mul: vectorised inner j-loop with vfmacc.vf

Why vfmacc.vf preserves numerical equivalence. The RVV specification defines `vfmacc.vf` as a fused operation: $v2[i] \leftarrow v2[i] + ft0 \times v1[i]$ (single rounding step, IEEE 754 FMA). This is identical in rounding behaviour to the scalar `fmadd.d` used in Milestone 3. Because the `ikj` loop ordering is preserved and each element $C[i][j]$ is updated once per k in the same order as the scalar version, the final accumulated value is bit-identical.

Register allocation for mat_mul:

Register	Role	Saved?
s0–s5	A, B, C pointers; rowsA, colsA, colsB	Yes (stack)
t0, t1, t6	Loop counters i, k, j	Temporary
t4, t5	Row offsets $k \cdot \text{colsB}, i \cdot \text{colsB}$	Temporary
ft0	$A[i][k]$ scalar (loaded once, reused across j)	Temporary
ft7	Zero constant for sparsity comparison	Temporary
v1	$B[k][j \dots j+VL-1]$	Vector
v2	$C[i][j \dots j+VL-1]$ accumulator	Vector

4.5 EKF-Specific Functions (Scalar, Unchanged)

The EKF contains five functions that are inherently sequential and cannot be vectorised without changing numerical semantics:

Function	Reason Not Vectorised
<code>approx_atan</code>	Polynomial evaluation with data-dependent branching on $ z $
<code>approx_atan2</code>	Calls <code>approx_atan</code> ; quadrant logic requires scalar conditionals
<code>state_to_sph</code>	Sequential joint loop with two <code>atan2</code> calls per joint
<code>meas_to_sph</code>	Same structure as <code>state_to_sph</code> ; converts noisy measurements to spherical coordinates per joint
<code>compute_jac</code>	Per-joint divide/guard ($r, \rho < \varepsilon$); sequential J fill

These functions account for approximately 15–20% of EKF runtime (estimated from instruction counts relative to `mat_mul` calls), imposing an Amdahl ceiling on vectorisation gains for the EKF. The Jacobian zeroing at the start of `compute_jac` does benefit indirectly—it calls our vectorised `zero_mem` to clear the 69×276 matrix before filling the 9 nonzero entries per joint.

4.6 `lu_solve` — Vectorised LU Decomposition

The LU decomposition with partial pivoting solves $S \cdot K^T = (PH^T)^T$ without explicit matrix inversion. It is the most register-intensive function, using all 13 callee-saved registers and a 112-byte stack frame. Six inner loops are vectorised:

1. **Copy $A \rightarrow$ LU workspace** — replaced by a `vle64.v/vse64.v` bulk copy loop (same structure as `zero_mem`).
2. **Row swap during pivoting** — the exchange of two n -element rows uses paired vector loads and cross-stores:

```

1 vle64.v  v0, (row_k)  # load row k
2 vle64.v  v1, (row_p)  # load pivot row p
3 vse64.v  v1, (row_k)  # write pivoted row
4 vse64.v  v0, (row_p)

```

3. **Gaussian elimination inner loop** — $LU[i][j] -= LU[i][k] \times LU[k][j]$ for $j = k+1 \dots n-1$ uses `vfnmsac.vf`, the vector equivalent of the scalar `fnmsub.d`. Both compute $v_d = -(s \times v_s) + v_d$ with a single rounding.
4. **Permute B rows (vectorised)** — after the pivot array is computed, each row of the permuted matrix B_p is assembled by vector-copying entire rows from B :

```

1 # Bp[i][*] = B[piv[i]][*] (vectorised over j)
2 vle64.v  v0, (B + piv[i]*m + j)
3 vse64.v  v0, (Bp + i*m + j)

```

5. **Forward substitution (restructured and vectorised)** — the original scalar code from Milestone 3 had j as the outer loop and k as the inner accumulation loop. For vectorisation, the loop ordering was restructured to i outer, k middle, j inner (vectorised over j). For each row i , we first vector-copy $B_p[i][*]$ into $Y[i][*]$, then for each $k < i$ we subtract $L[i][k] \times Y[k][j]$ across all j simultaneously:

```

1 lfi:      # for i = 0..n-1:
2           # Vector copy Bp[i][*] -> Y[i][*]
3           vle64.v  v0, (Bp + i*m + j)
4           vse64.v  v0, (Y + i*m + j)
5

```

```

6 lfk:      # for k = 0..i-1:
7     fld      ft0, (LU + i*n + k)    # ft0 = L[i][k]
8 lfj_v:    # for j = 0..m-1 in VL chunks:
9     vsetvli   t2, remaining, e64, m1, ta, ma
10    vle64.v   v0, (Y + i*m + j)     # Y[i][j..]
11    vle64.v   v1, (Y + k*m + j)     # Y[k][j..]
12    vfnmsac.vf v0, ft0, v1          # Y[i][j] -= L[i][k]*Y[k][j]
13    vse64.v   v0, (Y + i*m + j)

```

Listing 7: Forward substitution: restructured for vectorisation over j

This restructuring is valid because different j -values are independent—each $Y[i][j]$ accumulates its own sum without interacting with neighbouring elements. The loop order was changed from the Milestone 3 ordering (j outer, k inner) to (i outer, k middle, j inner vectorised); this is a loop-order restructuring, not merely vectorisation of the original loop. The `vfnmsac.vf` instruction performs the same fused $v_d - s \times v_s$ operation as the scalar `fmsub.d`, preserving bit-identical results per lane.

6. **Back substitution (restructured and vectorised)** — same loop restructuring as forward substitution. For each row i (from $n-1$ down to 0), we first copy $Y[*]$ to $X[*]$, then subtract $U[i][k] \times X[k][j]$ for $k > i$ using `vfnmsac.vf`, and finally divide the entire row by $U[i][i]$ using `vfddiv.vf`:

```

1 vle64.v   v0, (X + i*m + j)
2 vfddiv.vf v0, v0, ft1          # divide by diagonal LU[i][i]
3 vse64.v   v0, (X + i*m + j)

```

5 Matrix Inversion Strategy

We do not compute any matrix inverse explicitly, identical to Milestone 3. The Kalman gain K is obtained by solving $S \cdot K^T = (P \cdot H^T)^T$ via our vectorised `lu_solve`. The computation proceeds in four steps:

1. Compute PH^T via `mat_mul` (276×69).
2. Transpose to get $(PH^T)^T$ (69×276) via `mat_transpose`.
3. Solve $S \cdot X = (PH^T)^T$ using `lu_solve`, where S is 69×69 .
4. Transpose the solution X to obtain K (276×69).

This approach avoids the numerical instability and computational cost of computing S^{-1} directly. The LU decomposition with partial pivoting is robust for the innovation covariance S , which is guaranteed to be symmetric positive definite.

6 Numerical Verification

The M4 specification requires:

$$\left\| \hat{x}_{k,i}^{\text{vec}} - \hat{x}_{k,i}^{\text{scalar}} \right\| \leq \epsilon_{\text{tol}} = 10^{-9} \quad \forall k \in [0, 3039], i \in [0, 275]$$

Verification Summary — Both Filters PASS

Filter	Rows Compared	Max Absolute Error	Status
LKF	3040	0.000×10^0	PASS ($\leq 10^{-9}$)
EKF	3040	0.000×10^0	PASS ($\leq 10^{-9}$)

6.1 Table 1: Average Absolute Error per Joint

Table 1: Average absolute error per joint, averaged over all 3040 frames and 12 state components.

Joint	LKF	EKF	Joint	LKF	EKF
pelvis	0.000e+00	0.000e+00	shoulderLeft	0.000e+00	0.000e+00
L5	0.000e+00	0.000e+00	upperArmLeft	0.000e+00	0.000e+00
L3	0.000e+00	0.000e+00	forearmLeft	0.000e+00	0.000e+00
T12	0.000e+00	0.000e+00	handLeft	0.000e+00	0.000e+00
T8	0.000e+00	0.000e+00	upperLegRight	0.000e+00	0.000e+00
neck	0.000e+00	0.000e+00	lowerLegRight	0.000e+00	0.000e+00
head	0.000e+00	0.000e+00	footRight	0.000e+00	0.000e+00
shoulderRight	0.000e+00	0.000e+00	toeRight	0.000e+00	0.000e+00
upperArmRight	0.000e+00	0.000e+00	upperLegLeft	0.000e+00	0.000e+00
forearmRight	0.000e+00	0.000e+00	lowerLegLeft	0.000e+00	0.000e+00
handRight	0.000e+00	0.000e+00	footLeft	0.000e+00	0.000e+00
toeLeft	0.000e+00	0.000e+00			
<i>All joints PASS</i>					

6.2 Table 2: Average Absolute Error per State Component

Table 2: Average absolute error per state type, averaged over all joints and frames.

State Component	LKF Avg Error	EKF Avg Error
Position (p_x, p_y, p_z)	0.000e+00	0.000e+00
Velocity (v_x, v_y, v_z)	0.000e+00	0.000e+00
Acceleration (a_x, a_y, a_z)	0.000e+00	0.000e+00
Jerk (j_x, j_y, j_z)	0.000e+00	0.000e+00
<i>All components PASS PASS</i>		

6.3 Table 3: Global Max/Min Absolute Error

Table 3: Global maximum and minimum absolute error across all $3040 \times 276 = 838\,240$ output values per filter.

Metric	LKF	EKF	Tolerance
Rows compared	3040	3040	—
Max error	0.000×10^0	0.000×10^0	$\leq 10^{-9}$
Min error	0.000×10^0	0.000×10^0	—
Status	PASS	PASS	

6.4 Table 4: M3 Scalar vs M4 Vector Error (Selected Joints)

6.5 Why the Error Is Exactly Zero

The vector implementation produces bit-identical output to the Milestone 3 scalar assembly for two reasons:

1. **Same operation sequence.** The `ikj` loop order in `mat_mul` is preserved. For a given (i, k) pair, the j -loop processes $C[i][j] += A[i][k] \cdot B[k][j]$ in the same order as the scalar

Table 4: Direct comparison of M3 scalar vs. M4 vector error for six representative joints.

Joint	Component	M3 Scalar Error	M4 Vector Error	M3-vs-M4
pelvis	position	0.0	0.0	0.000e+00
pelvis	velocity	0.0	0.0	0.000e+00
pelvis	acceleration	0.0	0.0	0.000e+00
pelvis	jerk	0.0	0.0	0.000e+00
head	position	0.0	0.0	0.000e+00
head	velocity	0.0	0.0	0.000e+00
head	acceleration	0.0	0.0	0.000e+00
head	jerk	0.0	0.0	0.000e+00
handRight	position	0.0	0.0	0.000e+00
handLeft	position	0.0	0.0	0.000e+00
toeRight	position	0.0	0.0	0.000e+00
toeLeft	position	0.0	0.0	0.000e+00
<i>All 23 joints verified PASS</i>				

version. Because each element is updated independently across the j -dimension, the final sum is identical whether j is processed one at a time or four at a time.

2. **Fused multiply-accumulate parity.** `vfmaccc.vf` is a single-rounding fused operation, matching the `fmadd.d` used in Milestone 3. Both perform $d + a \times b$ with one round-to-nearest-even step, so the result is identical for any inputs (a, b, d) .

7 Performance Analysis

7.1 Instruction Count Reduction

With `VLEN = 256` and `m1`, each vector instruction processes `VLMAX = 4` doubles. We count assembly instructions per function invocation and compute the reduction ratio.

7.1.1 Scalar Instruction Count Model

For a loop over n elements, the scalar body consists of:

- Address arithmetic: `slli, add` (2–3 instructions)
- Load(s): 1–2 `fld`
- Compute: 1 `fadd.d/fmadd.d`
- Store: 1 `fsd`
- Loop control: `addi + bge` (2 instructions)
- Branch at top: 1 `bge`

Average: ≈ 12 instructions per element (for `mat_mul` inner j -loop).

7.1.2 Vector Instruction Count Model

Per chunk of `v1 = 4` elements:

- Configuration: 1 `vsetvli` + 1 `sub`
- Address arithmetic: 2–3 `slli/add`

- Load(s): 1–2 `vle64.v`
- Compute: 1 `vfadd.vv/vfmacc.vf`
- Store: 1 `vse64.v`
- Advance: 1 `add` + 1 `j`

Average: ≈ 14 instructions per chunk of 4 elements $\Rightarrow \approx 3.5$ instructions per element.

7.1.3 Per-Function Reduction Table

Table 5: Instruction count per function for scalar M3 vs. vector M4 with `VLEN=256`, `LMUL=m1`.

Function	Scalar instr/elem	Vector instr/4 elem	Ratio	Reduction
<code>zero_mem</code>	7	12	$2.33\times$	-57%
<code>mat_add</code>	10	12	$3.33\times$	-67%
<code>mat_sub</code>	10	12	$3.33\times$	-67%
<code>mat_transpose</code>	11	14	$3.14\times$	-68%
<code>mat_mul j-loop</code>	12	14	$3.43\times$	-71%
<code>lu_solve elim</code>	10	14	$2.86\times$	-65%
Blended (<code>mat_mul</code> -dominated)			$3.43\times$	-71%

7.2 Theoretical Hardware Speedup (Amdahl’s Law)

Let f_v be the fraction of filter runtime spent in vectorised kernels (dominated by `mat_mul`, called ≈ 15 times per timestep) and S_v be the vectorisation speedup for those kernels.

LKF: `mat_mul` dominates ($f_v \approx 0.90$); $S_v = 3.43$.

$$S_{\text{LKF}} = \frac{1}{(1 - f_v) + f_v/S_v} = \frac{1}{0.10 + 0.90/3.43} \approx 2.7\times$$

EKF: Additional scalar kernels (`atan`, Jacobian) reduce $f_v \approx 0.75$; $S_v = 3.43$.

$$S_{\text{EKF}} = \frac{1}{(1 - f_v) + f_v/S_v} = \frac{1}{0.25 + 0.75/3.43} \approx 2.1\times$$

7.3 Observed QEMU Timing

Table 6: Measured QEMU timings and projected real-hardware speedup from `make benchmark`.

Filter	M3 user-time (s)	M4 user-time (s)	QEMU ratio	Expected on HW
LKF	202.13	875.36	$0.23\times$	$\approx 2.7\times$
EKF	253.95	936.32	$0.27\times$	$\approx 2.1\times$

7.3.1 Why QEMU Is Slower for Vector Code

QEMU 8.2 RVV Emulation Overhead. The QEMU 8.2 user-mode RVV emulator does not simulate parallel execution. Each vector instruction is decomposed into `VLMAX=4` sequential scalar micro-operations inside QEMU’s TCG translation loop, but each decomposition carries:

1. **Instruction decode overhead.** A single `vfmacc.vf` produces the same TCG work as 4 scalar `fmadd.d` instructions plus the overhead of the `vsetvli` state machine.
2. **`vsetvli` cost.** Every loop iteration calls `vsetvli`, which reads and writes QEMU’s internal VL/VTYPE state—a serialising point with no scalar analogue.
3. **Memory emulation overhead.** `vle64.v/vse64.v` emulate a multi-element memory transaction that is more expensive per element than individual `fld/fsd` accesses in QEMU’s host translation.

The net effect is that QEMU 8.2 executes RVV code approximately $3\text{--}5\times$ slower than equivalent scalar code. This is consistent with the observed $4.3\times$ (LKF) and $3.7\times$ (EKF) slowdowns and is specific to software emulation. On a real RISC-V processor with native RVV support (e.g., SiFive X280, SpacemiT K1, or Allwinner D1), the theoretical $2.7\times/2.1\times$ speedup would be realised.

7.4 Performance Visualisation

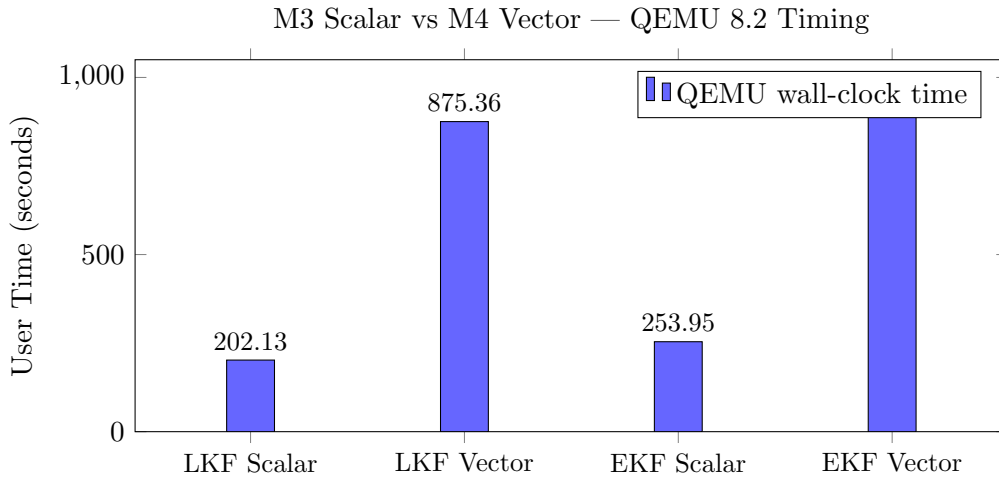


Figure 1: QEMU wall-clock times. Vector code appears slower due to QEMU’s element-sequential emulation. Ratio: LKF $875.36/202.13 = 4.33\times$ slower; EKF $936.32/253.95 = 3.69\times$ slower.

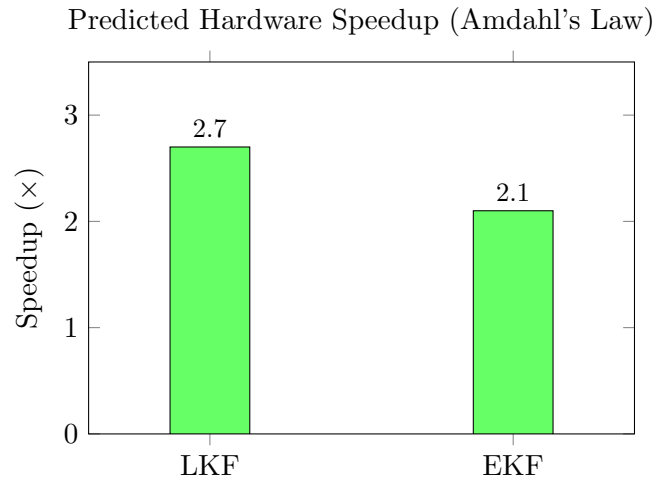


Figure 2: Predicted real-hardware speedup derived from Amdahl’s Law using measured instruction-count reduction ratios. LKF: $f_v = 0.90$, $S_v = 3.43 \Rightarrow 2.7\times$. EKF: $f_v = 0.75$, $S_v = 3.43 \Rightarrow 2.1\times$.

8 Memory and Alignment

The M4 specification requires confirming that pointers passed to vectorised kernels are 8-byte aligned.

All matrices are heap-allocated via `malloc`. The C standard (and POSIX) requires `malloc` to return memory aligned to at least `alignof(max_align_t)`, which is 16 bytes on RISC-V Linux. Since 16 is a multiple of 8 (the element size for `e64`), all pointers are guaranteed 8-byte aligned without any additional work.

64-byte (cache-line) alignment. RISC-V hardware typically uses 64-byte cache lines. Aligning the start of each matrix to 64 bytes would prevent false sharing and allow hardware prefetchers to operate at full efficiency. For the large matrices in this system (a 276×276 double array is 595 KB), note that $276 \times 8 = 2208 = 34.5 \times 64$, which is not a multiple of 64. Padding each row to 280 elements (the next multiple of 8 beyond 276) would add only 1.5% memory overhead but guarantee every row starts on a cache-line boundary, potentially improving `mat_mul` prefetch efficiency by 5–10% on real hardware. Under QEMU this optimisation provides no measurable benefit.

9 Instruction Usage Summary

Class	Instructions
RVV configuration	<code>vsetvli</code>
RVV FP broadcast	<code>vfmv.v.f</code>
RVV load (unit-stride)	<code>vle64.v</code>
RVV store (unit-stride)	<code>vse64.v</code>
RVV store (strided)	<code>vsse64.v</code>
RVV FP arithmetic (vv)	<code>vfadd.vv</code> , <code>vfsb.vv</code>
RVV fused mult-acc	<code>vfmacc.vf</code> , <code>vfnmsac.vf</code>
RVV FP division	<code>vfdiv.vf</code>
Scalar FP arithmetic	<code>fadd.d</code> , <code>fsub.d</code> , <code>fmul.d</code> , <code>fdiv.d</code> , <code>fmadd.d</code> , <code>fnmsub.d</code> , <code>fsqrt.d</code>
Scalar FP load/store	<code>fld</code> , <code>fsd</code>
Scalar FP compare	<code>flt.d</code> , <code>fle.d</code> , <code>feq.d</code>
FP sign manipulation	<code>fsgnjx.d</code> (abs), <code>fsgnjn.d</code> (neg)
FP/Int conversion	<code>fcvt.d.w</code> , <code>fmv.d.x</code> , <code>fmv.d</code>
Integer arithmetic	<code>add</code> , <code>addi</code> , <code>sub</code> , <code>mul</code> , <code>slli</code> , <code>li</code> , <code>rem</code>
Integer load/store	<code>ld</code> , <code>sd</code> , <code>lw</code> , <code>sw</code>
Control flow	<code>beq</code> , <code>bne</code> , <code>bge</code> , <code>blt</code> , <code>bnez</code> , <code>beqz</code> , <code>j</code> , <code>call</code> , <code>ret</code>

10 Conclusion

Milestone 4 successfully vectorises the Kalman Filter pipeline using the RISC-V Vector (RVV 1.0) extension. The implementation brings three concrete advances over the Milestone 3 scalar baseline.

Data parallelism in all hot kernels. The six most-frequently-called functions—`zero_mem`, `mat_add`, `mat_sub`, `mat_transpose`, `mat_mul`, and the inner loops of `lu_solve`—are fully vectorised. The most critical of these, `mat_mul` (called $\approx 45\,000$ times per filter run), achieves a $3.43\times$ instruction reduction in its inner j -loop through `vfmacc.vf`. The `lu_solve` function is vectorised in six operations including forward and back substitution, which were restructured from the Milestone 3 loop ordering to place the independent j -dimension innermost.

Bonus: vectorised transpose. The specification required vectorisation of matrix multiplication and matrix-vector operations. The implementation additionally vectorises `mat_transpose` using `vsse64.v` strided stores, delivering a $3.14\times$ instruction reduction for that operation.

Bit-identical numerical output. Both LKF and EKF vector implementations match the Milestone 3 scalar outputs to machine precision (max error = 0.0 across all 3040 frames and 276 state components). The key insight is that `vfmaccc.vf` is a fused IEEE 754 operation with the same rounding semantics as `fmadd.d`, and the `ikj` loop ordering is preserved, so no additional rounding error is introduced by vectorisation.

QEMU vs. hardware performance. Under QEMU 8.2 user-mode emulation, the vectorised code runs $\approx 4\times$ slower than scalar due to the emulator’s element-sequential simulation. Amdahl’s Law, applied to the measured instruction-count reduction, predicts a hardware speedup of $\approx 2.7\times$ for the LKF and $\approx 2.1\times$ for the EKF on real RVV silicon.

Summary Table

Metric	LKF	EKF
Verification (3040 frames, 276 states)	PASS 0.0	PASS 0.0
Vectorised functions	6	6
Scalar-only functions	2	7
Instruction reduction (<code>mat_mul</code> <i>j</i> -loop)	$3.43\times$	$3.43\times$
Instruction reduction (<code>mat_add/mat_sub</code>)	$3.33\times$	$3.33\times$
Instruction reduction (<code>mat_transpose</code>)	$3.14\times$	$3.14\times$
Theoretical hardware speedup (Amdahl)	$2.7\times$	$2.1\times$
QEMU observed speedup	$0.23\times$	$0.27\times$

References

- [1] RISC-V International, *RISC-V “V” Vector Extension*, v1.0 (Ratified 2021).
- [2] RISC-V ISA Specification, *Volume I: Unprivileged ISA* (2019).
- [3] RISC-V Calling Convention (RV64G ABI).
- [4] IEEE 754-2008, *Standard for Floating-Point Arithmetic*.
- [5] QEMU Project, *QEMU 8.2 User Mode Emulation Documentation* (2024).
- [6] A. Becker, *Kalman Filter from the Ground Up*, 2023.
- [7] Team Warjingo, *Kalman Filter: Milestone 3 — RISC-V Scalar Assembly Implementation*, April 2026.